



Student Name:A.Sathwika Reddy

Reg. No.: 192525274

Course Code: CSA0619

Slot:D

Course Name : Design and Analysis of Algorithms

Course Faculty: Dr F. Mary Harin Fernandez

CO2_ASSESSMENT TOOL 3

Aim :To debug the given brute force Closest Pair algorithm by eliminating self-comparisons and duplicate distance calculations, while preserving the brute force approach. The objective is to correctly compute the minimum distance between distinct pairs of points and analyze the algorithm's time complexity.

Algorithm

1. Initialize the minimum distance as infinity.
2. Iterate through each point using index i .
3. Compare it only with the points having index greater than i ($j = i + 1$).
4. Compute the Euclidean distance between the two points.
5. If the computed distance is smaller than the current minimum distance, update it.
6. Continue until all unique pairs have been checked.
7. Return the minimum distance.

OUTPUT:

A screenshot of a terminal window with a dark background. The title bar at the top shows standard window controls and the text 'input'. The terminal content displays 'Minimum Distance = 1.41' in white. Below this, two lines of green text are visible: '...Program finished with exit code 0' and 'Press ENTER to exit console.' followed by a cursor icon.

```
Minimum Distance = 1.41

...Program finished with exit code 0
Press ENTER to exit console.
```

Analysis

Time Complexity:

- Outer loop runs **n** times.
- Inner loop runs:
 - $(n-1) + (n-2) + \dots + 1$
 - $= n(n-1)/2$

Therefore,

Time Complexity:

$O(n^2)$

- Only a few variables (`min_dist`, `i`, `j`, `dist`) are used.

Space Complexity:

$O(1)$

Result

The brute force Closest Pair algorithm was successfully debugged by:

- Removing self-comparisons.

- Eliminating duplicate pair evaluations.
- Comparing only unique point pairs exactly once.

The corrected implementation accurately computes the minimum Euclidean distance while maintaining the brute force strategy. The optimized version performs $\mathbf{n(n-1)/2}$ distance calculations instead of $\mathbf{n^2}$, improving efficiency without changing the overall $\mathbf{O(n^2)}$ time complexity and $\mathbf{O(1)}$ space complexity.